

1. Variables

Tipos de variables

Hay 3 tipos más comunes de variables:

Numéricas: almacenan números, dentro de ellas podemos encontrar subtipos `int`, `float`, `long`, `complex`.

Texto: son las clásicas variables que almacenan cadenas de texto.

Booleanas: almacenan un valor `True` o `False`.

Declarando variables

En Python, al declarar una variable **no** necesitamos especificar el tipo de dato que vamos a almacenar en ella, como sí es necesario en otros lenguajes de programación.

```
caja1 = "Hola pin pon"  
caja2 = 18  
caja3 = 18.1  
caja4 = True
```

2. Operadores

Operadores aritméticos

Operador	Función	Ejemplo	Resultado
+	Sumar	2 + 2	4
-	Restar	3 - 2	1
*	Multiplicar	2 * 2	4
/	Dividir	4 / 2	2
%	Módulo: devuelve el resto de la división	4 % 2	0
**	Exponente: exponencial de un número	3 ** 2	9
//	División entera: devuelve el entero de la división	8 // 4	2

Operadores de comparación

Operador	Función	Ejemplos	Resultado
==	True si dos valores son exactamente iguales	2 == 2 2 == 3	True / False
!=	True si dos valores son diferentes	2 != 5 2 != 2	True / False
>	True si el valor de la izquierda es mayor	4 > 2 1 > 2	True / False
<	True si el valor de la izquierda es menor	1 < 2 4 < 2	True / False
>=	True si es mayor o igual	4 >= 2 1 >= 2 4 >= 4	True / False / True
<=	True si es menor o igual	4 <= 6 1 <= 0 4 <= 4	True / False / True

Operadores de asignación

Operador	Función	Ejemplo	Resultado
=	Asigna un valor a un elemento (variable, lista, diccionario, tupla, etc.)	a = 2 + 2	«a» vale 4
+=	El elemento es igual a la suma consigo mismo. Se usa como contador	b += 1	b = b + 1
-=	El elemento es igual a la resta consigo mismo. Contador negativo	b -= 1	b = b - 1

Operador	Función	Ejemplo	Resultado
<code>*</code>	El elemento es igual a la multiplicación consigo mismo	<code>b * = 2</code>	<code>b = b * 2</code>
<code>/</code>	El elemento es igual a la división consigo mismo	<code>b /= 2</code>	<code>b = b / 2</code>
<code>%</code>	El elemento es igual al módulo (resto) consigo mismo	<code>b %= 2</code>	<code>b = b % 2</code>
<code>**</code>	El elemento es igual al resultado de la exponente consigo mismo	<code>b ** = 2</code>	<code>b = b ** 2</code>

Operadores lógicos

Operador	Función	Ejemplos	Resultado
<code>and</code>	True solo si todos los elementos son True. Si no, False	True and True False and False True and False False and True	True / False / False / False
<code>or</code>	True si algún elemento es True. Si no, False	True or True False or False True or False False or True	True / False / True / True
<code>not</code>	Operador unario de negación: invierte el valor booleano	not True not False	False / True

Operadores especiales

Operador	Función	Ejemplo	Resultado
<code>in</code>	True si un elemento se encuentra dentro de otro	<code>a = [3, 4]</code> <code>3 in a</code>	True (3 está en a)
<code>not in</code>	True si un elemento no se encuentra dentro de otro	<code>a = [3, 4]</code> <code>5 in a</code>	True (5 no está en a)
<code>is</code>	True si los elementos son exactamente iguales	<code>x = 10</code> <code>y = 10</code> <code>x is y</code>	True (mismo valor)
<code>is not</code>	True si los elementos no son exactamente iguales	<code>x = 10</code> <code>y = 111</code> <code>x is not y</code>	True (valores distintos)

3. Condicionales

Condicional if

Para utilizar el if basta con agregarlo en el siguiente orden:

IF + (Condición) + **:**

Y debajo de los dos puntos, indentada (con la tecla Tab), la instrucción a realizar en caso de que la condición se cumpla.

```
a = 2 + 2
if a == 4: # condición: si a es exactamente cuatro, entonces:
    print("A es igual a cuatro") # Imprimir
# Resultado: "A es igual a cuatro"
```

Condicional if - else

Luego de una sentencia **if** podemos agregar un **else** («si no») para que se ejecute otro código en caso de que la condición no se cumpla.

```
a = 2 + 3
if a == 4: # condición: si a es exactamente cuatro, entonces:
    print("A es igual a cuatro") # Imprimir
else:
    print("No se cumple la condicion")
# Resultado: "No se cumple la condicion"
```

En este caso se imprime «No se cumple la condición» porque, al no cumplirse el if, el intérprete va directamente al else (si existe) y lo ejecuta.

Condicionales if - elif - else

Se pueden agregar tantos **elif** como sea necesario para comprobar condiciones adicionales antes de llegar al **else** final.

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
#
# operadores.py
#
a = 2 + 3
if a == 4: # condición: si a es exactamente cuatro, entonces:
    print("A es igual a cuatro") # Imprimir
elif a == 5:
    print("A es igual a cinco")
elif a == 6:
    print("A es igual a seis")
else:
    print("No se cumple la condición")
# Resultado: "A es igual a cinco"
```

4. Bucles for y while

¿Cuándo usar for y cuándo usar while?

- Si se sabe el número exacto de veces que se repite un proceso, se debe usar for (por ejemplo, el cálculo de un factorial: desde N hasta 1).
- Si no se sabe cuándo va a terminar la repetición —depende de una condición que no sabemos cuándo se cumplirá— se debe usar while (por ejemplo, un combate donde se repiten turnos hasta que alguno de los dos contendientes pierda toda su vida).

Bucle for

Un bucle **for** establece la variable iteradora en cada valor de una lista, arreglo o cadena proporcionada, y repite el código en el cuerpo del bucle para cada valor de la variable iteradora. A diferencia de otros lenguajes como JavaScript o C, en Python el **for** funciona de manera distinta: recorre directamente los elementos de una secuencia.

```
# Ejemplo bucle for
for i in [1, 2, 3, 4]:
    print(i, end="-", " ") # prints: 1-, 2-, 3-, 4-,
```

También se puede incluir una lógica más compleja en el cuerpo de un bucle for:

```
# Ejemplo más complejo
for i in [1, 3, 5, 7, 9]:
    x = i**2 - (i-1)*(i+1)
    print(x, end="-", " ") # prints 1, 1, 1, 1, 1
```

Ejemplo con lista de cursos (**cursos = ["IA", "Big Data", "SQL", "Python", "SEO"]**):

```
cursos = ["IA", "Big Data", "SQL", "Python", "SEO"]
for x in cursos:
    print(x)
```

La función range()

Cuando los valores de un arreglo para nuestro bucle **for** son secuenciales, podemos usar la función **range()** en lugar de escribir el contenido completo del arreglo.

```
range(parar)
range(inicio, parar[, paso])
```

- inicio: primer valor del rango. Si se llama a range() con un solo argumento, Python asume inicio = 0.
- parar: límite superior del rango. Este valor NO está incluido en el rango.
- paso (opcional): controla el incremento entre los valores del rango. Por defecto, paso = 1.

Ejemplo con un argumento (rango 0 a 4):

```
for i in range(5):
    print(i, end="-", " ") # prints: 0, 1, 2, 3, 4,
```

Ejemplo con dos argumentos (inicio = -1):

```
for i in range(-1, 5):
    print(i, end="-", " ") # prints: -1, 0, 1, 2, 3, 4,
```

Ejemplo con tres argumentos (rango de -1 a 5, paso = 2):

```
for i in range(-1, 5, 2):  
    print(i, end="-", ") # prints: -1, 1, 3,
```

Bucle while

Los bucles **while** se utilizan para repetir una operación hasta que la condición especificada en su declaración sea falsa. Son especialmente útiles cuando se desconoce el número exacto de instancias en las que debe ejecutarse el bucle, ya que seguirá iterando hasta que se cumplan sus condiciones (por ejemplo, la búsqueda de información en grandes conjuntos de datos, deteniéndose automáticamente al encontrar el dato deseado).

```
i = 1  
print("Insertando Pin...")  
while i < 3:  
    print("ERROR. Sigue intentando insertar el PIN... Tienes", 3 - i, "intentos.")  
    i += 1  
print("Has superado los 3 intentos. Llamando a @policia...")
```

Resumen

- Los bucles for repiten un bloque de código para todos los valores de una lista, arreglo, cadena o range().
- Podemos usar la función range() para simplificar la escritura de un bucle for, especificando el valor de parar (obligatorio) y opcionalmente inicio y paso.
- Los bucles while repiten una operación hasta que la condición deja de cumplirse; son ideales cuando no se conoce de antemano el número de repeticiones.

5. Listas y tuplas

¿Qué son las listas en Python?

Las listas son colecciones ordenadas y modificables (mutables) de elementos, que pueden ser de distintos tipos.

Diferencias y similitudes entre listas y tuplas

Listas	Tuplas
Se definen con corchetes []	Se definen con paréntesis ()
Son mutables: se pueden modificar tras su creación	Son inmutables: no se pueden modificar tras su creación
Permiten añadir, eliminar y modificar elementos	No permiten añadir, eliminar ni modificar elementos
Se recorren e indexan igual que las tuplas	Se recorren e indexan igual que las listas

Accediendo a listas y tuplas: indexado y slicing

Se puede acceder a un elemento concreto mediante su índice (empezando en 0), o a un subconjunto mediante slicing (`lista[inicio:fin]`). También se admiten índices negativos para contar desde el final de la lista.

```
lista = ["a", "b", "c", "d", "e"]
print(lista[0])      # "a"
print(lista[-1])     # "e" (último elemento)
print(lista[1:3])    # ["b", "c"]
```

Métodos de listas (modificación)

Estos métodos modifican la lista original directamente.

Tabla resumen de funciones y métodos de listas

Función	Qué hace	Ejemplo	Resultado
sort()	Ordena la lista de manera ascendente	<code>l = [2,1,4]; l.sort()</code>	<code>print(l) → [1, 2, 4]</code>
clear()	Vacía la lista	<code>l = [2,1,4]; l.clear()</code>	<code>print(l) → []</code>
max()	Devuelve el máximo de los elementos	<code>l = [2,1,4]; max(l)</code>	Devuelve 4
min()	Devuelve el mínimo de los elementos	<code>l = [2,1,4]; min(l)</code>	Devuelve 1
append()	Añade un único elemento a la lista	<code>l = [2,1,4]; l.append(5)</code>	<code>l = [2, 1, 4, 5]</code>
extend()	Añade varios elementos (falla con uno solo)	<code>l = [2,1,4]; l.extend([5,6])</code>	<code>l = [2, 1, 4, 5, 6]</code>
index()	Devuelve la primera posición del elemento buscado	<code>l = [2,1,4]; l.index(1)</code>	Devuelve 1 (primera vez que aparece)
count()	Cuenta apariciones de un elemento (puede dar 0)	<code>l = [1,1,4]; l.count(1)</code>	El resultado da 2

Función	Qué hace	Ejemplo	Resultado
reverse()	Invierte los elementos (equivale a lista[::-1])	l = [1,1,4]; l.reverse() ó l[::-1]	El resultado da [4, 1, 1]

Eliminar elementos de una lista

Se pueden usar `del`, `remove()` o `pop()` según se quiera eliminar por índice, por valor, o extraer el último elemento (u otro por índice) devolviéndolo.

El resto Recursivo (*args)

Python permite el desempaquetado de listas y el uso de un «resto» para capturar múltiples valores restantes al desestructurar una secuencia.

6. Diccionarios

¿Qué es un diccionario en Python?

Los diccionarios son estructuras de datos de Python que permiten almacenar datos en pares clave-valor. Cada elemento de un diccionario está compuesto por una clave única y su respectivo valor; la clave puede ser de cualquier tipo inmutable, y el valor puede ser cualquier objeto válido en Python. Se definen usando llaves {}, y los pares clave-valor se separan por puntos y comas.

```
mi_diccionario = {  
    "nombre": "Juan",  
    "curso": "sql",  
    "nota": 7.2  
}
```

Cómo funcionan los diccionarios en Python

Internamente, los diccionarios se implementan mediante una estructura de datos llamada tabla hash, que permite una búsqueda rápida y eficiente de los elementos del diccionario, incluso en un gran número de elementos.

Cuando se agrega un nuevo par clave-valor a un diccionario, Python calcula un valor hash único para la clave proporcionada; ese valor se utiliza para determinar la posición donde se almacenará el valor en la tabla hash. Cuando se busca un elemento en el diccionario, Python calcula de nuevo el hash de la clave de búsqueda y lo busca en la posición correspondiente.

Nota: el hash es un valor único, por eso, al igual que en una lista tradicional, si no hay palabra (clave) Python indica rápido que no habrá definido (valor).

Operaciones básicas en diccionarios

Los diccionarios en Python admiten una variedad de operaciones básicas que permiten manipular y trabajar con los elementos dentro de ellos: agregar elementos, eliminar elementos, actualizar valores y comprobar si una clave existe en el diccionario.

Añadir elemento en diccionario

Para agregar un nuevo par clave-valor a un diccionario, se puede utilizar la sintaxis:

```
mi_diccionario["nueva_clave"] = "nuevo_valor".
```

Cómo eliminar un elemento del diccionario

Para eliminar un elemento del diccionario, se puede utilizar la palabra clave `del` seguida de la clave que se desea eliminar: `del mi_diccionario["clave_a_eliminar"]`.

Actualizar valores en diccionario

Para actualizar el valor de un elemento existente en el diccionario, simplemente se asigna un nuevo valor a la clave correspondiente: `mi_diccionario["clave_existente"] = "nuevo_valor"`.

Comprobar si existe una clave en el diccionario

Para verificar si una clave está presente en el diccionario, se puede utilizar el operador `in`:

```
if "clave" in mi_diccionario:  
    # hacer algo
```

Funciones comunes en Python para diccionarios

`get()` para obtener el valor: el método `get()` se utiliza para acceder al valor asociado a una clave en un diccionario de manera segura, es decir, sin generar errores si la clave no existe en el diccionario. Además, este método permite establecer un valor por defecto en caso de que la clave no se encuentre en el diccionario.

```
edades = {"Juan": 30, "Maria": 25, "Carlos": 35}

nombre = "Ana"
edad = edades.get(nombre)

if edad is None:
    print(nombre, "no está en el diccionario de edades.")
else:
    print(nombre, "tiene", edad, "años.")
```

Este enfoque útil para evitar errores de tipo `KeyError` que ocurrirían si intentáramos acceder directamente a una clave que no existe en el diccionario sin usar `get()`. Con `get()`, podemos proporcionar un valor por defecto o manejar de manera adecuada la ausencia de la clave en el diccionario.

Tamaño del diccionario: `len()`

Siguiendo con el ejemplo anterior, podríamos utilizar `len()` antes de recorrer el número de elementos de nuestro diccionario.

```
print(len(edades))
```

`len()` es una función útil para determinar rápidamente el tamaño de un diccionario, lo que puede ser especialmente útil cuando se necesitan operaciones que dependen del número de elementos en el diccionario, como en bucles o validaciones.

Diccionarios en Python: métodos más usados

Método	Descripción
<code>dict.keys()</code>	Devuelve la lista de las claves del diccionario.
<code>dict.values()</code>	Devuelve la lista de los valores del diccionario.
<code>dict.items()</code>	Devuelve una lista de tuplas (clave, valor).
<code>dict.get(clave)</code>	Devuelve el valor asociado a una clave, o <code>None</code> (o un valor por defecto) si no existe.
<code>dict.pop(clave)</code>	Elimina una clave y devuelve su valor asociado.
<code>dict.update(otro_dict)</code>	Actualiza el diccionario con los pares clave-valor de otro diccionario.
<code>len(dict)</code>	Devuelve la cantidad de elementos en el diccionario.

En resumen, estos son algunos de los métodos más comunes en los diccionarios de Python, pero hay muchas más disponibles para realizar diferentes operaciones y manipulaciones de datos almacenados en la memoria del programa.

7. Excepciones (try / except)

Sintaxis de try y except en Python

El bloque `try` permite ejecutar código que podría producir un error, mientras que `except` captura ese error e indica qué hacer en caso de que ocurra, evitando que el programa se detenga abruptamente.

```
try:
    # código que puede fallar
    resultado = 10 / 0
except ZeroDivisionError:
    print("No se puede dividir entre cero")
```

Cómo manejar múltiples excepciones en Python

Se pueden encadenar varios bloques `except` para capturar distintos tipos de error, cada uno con su propio tratamiento específico.

Cómo enfrentar TypeError en Python

Un `TypeError` se produce cuando se intenta realizar una operación sobre un tipo de dato incompatible.

Cómo manejar ValueError en Python

Un `ValueError` ocurre cuando una función recibe un argumento del tipo correcto pero con un valor inapropiado.

Cómo manejar KeyError en Python

Un `KeyError` se produce al intentar acceder a una clave que no existe en un diccionario. Se puede prevenir usando el método `get()` en lugar del acceso directo con corchetes.

Finalmente, se puede usar un bloque `finally` junto a `try/except` para ejecutar código de limpieza que debe correr siempre, ocurra o no una excepción, así como `raise` para lanzar excepciones propias cuando sea necesario.